

Coding & Solution

Date	1 October 2026
Team ID	SWTID-2026-3138
Project Name	ComicCraft - AI Comic Story Creator using Gemini Models
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	[Paste your GitHub / GitLab link here]
Programming Language(s)	Python (backend, AI integration); HTML, CSS and Jinja2 (frontend templates)
Framework(s) Used	FastAPI, Uvicorn (ASGI server), Jinja2Templates, Pydantic; Google Generative AI (Gemini 1.5 Flash and Pro); Hugging Face Diffusers + PyTorch (Stable Diffusion v1.5); FPDF; Pillow; python-dotenv
Key Features Implemented	1. Input form: story prompt, character name, setting, tone, art style 2. 5-panel outline generation (Gemini 1.5 Flash) - generate_outline() 3. Narration and dialogue generation (Gemini 1.5 Pro) - generate_story() 4. Panel image generation (Stable Diffusion v1.5) - generate_image() 5. Layout building - build_comic_layout() 6. Panel-by-panel comic preview page 7. Timestamped PDF export (FPDF) and export success page 8. JSON API (/generate-comic/json) and /test-image route; error handling with HTTPException and try-except
Pending / Incomplete Features	- User accounts, profiles and saved comic library - Editing or regenerating a single panel - Progress indicator while the AI models run - Character consistency across panels - More export options (images, share link) and multi-page story arcs - Cloud deployment (currently local only)
Setup / Run Instructions	1. Install Python and pip; create a virtual environment: python -m venv env 2. Activate it: env\Scripts\activate (Windows) or source env/bin/activate (macOS/Linux) 3. Install dependencies: pip install -r requirements.txt 4. Create a .env file with GEMINI_API_KEY=your_key and HF_API_KEY=your_key 5. From the project root run: uvicorn app.main:app --reload 6. Open http://127.0.0.1:8000 for the app and http://127.0.0.1:8000/docs for API testing

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The project is split into separate modules (gemini_flash.py, gemini_pro.py, image_generator.py, layout_builder.py, exporters.py, routes.py, main.py), each with one clear job. Functions have docstrings and inline comments. Gemini output is validated (JSON parsing and key checks) and routes use try-except with HTTPException. API keys are kept in a .env file, not in the code. Stable Diffusion image generation is slow on CPU, so a GPU is recommended for faster results.